SEGUNDO PARCIAL PRÁCTICO DE LABORATORIO (CORTE 2)

1. Contexto del Problema y Estructura del Examen

El Banco de Preguntas Saber PRO requiere integrar en una arquitectura de microservicios distribuida las funcionalidades de creación estructurada de reactivos, control de ciclo de vida con patrones de diseño comportamentales/estructurales y persistencia limpia desacoplada mediante Spring Data JPA.

La evaluación se divide en tres componentes prácticos integrados que deben desarrollarse y probarse durante el tiempo reglamentario de 80 minutos:

Componente	Tema / Origen (Taller)	Enfoque Técnico	Ponderación
Punto 1 (30%)	Patrones de Diseño GoF (Taller 08)	Patrón Builder (construcción de reactivo) y Patrón State (transición de estados sin condicionales).	**24 min**
Punto 2 (40%)	Microservicios REST y AMQP (Taller 09)	REST Síncrono (Question → Validation) y AMQP Asíncrono con RabbitMQ (SubmitQuestionForReview).	**32 min**
Punto 3 (30%)	Persistencia JPA Limpia (Taller 10)	Entidades @Entity en paquete record, QuestionDomainMapper bidireccional y repositorio JPA en H2.	**24 min**

2. Requerimientos Prácticos Evaluados

Punto 1. Patrones de Diseño GoF: Builder y State (Valor: 30%)

1.1. Patrón Builder (Creación de Pregunta): Implementar la interfaz IQuestionBuilder y la clase QuestionBuilder para ensamblar de forma fluida una pregunta de selección múltiple con 4 opciones de respuesta y 1 respuesta correcta identificada. El método build() debe validar la estructura del reactivo antes de instanciar el objeto de dominio.

1.2. Patrón State (Ciclo de Vida de la Pregunta): Implementar el control de transiciones de estado utilizando la interfaz IQuestionState y los estados concretos DraftState (Borrador) y PendingReviewState (Pendiente de Revisión). Está estrictamente prohibido usar estructuras condicionales (if/else o switch) para gestionar la transición.

Punto 2. Comunicación Síncrona REST y Asíncrona con RabbitMQ (Valor: 40%)

2.1. Comunicación Síncrona REST (Question → Validation Service): Al recibir una solicitud POST /questions en Question Service (:8083), este debe invocar de manera síncrona mediante RestTemplate al microservicio Validation Service (:8082) en POST /validate. Si la justificación del reactivo es válida (longitud > 15 caracteres), se guarda en estado DRAFT y responde HTTP 201 Created. Si la validación falla, responde HTTP 400 Bad Request sin guardar el registro.

2.2. Comunicación Asíncrona AMQP (RabbitMQ): Al invocar el envío a revisión mediante POST /questions/{id}/submit, Question Service debe publicar el comando SubmitQuestionForReview en la exchange saberpro.direct.exchange (routing key: question.submit) y retornar de inmediato la respuesta HTTP 202 Accepted con un operationId. El microservicio Lifecycle Service (:8084) consume el mensaje mediante @RabbitListener en question.submit.review.queue y efectúa la transición a PENDING_REVIEW.

Punto 3. Persistencia Relacional JPA y Arquitectura Limpia (Valor: 30%)

3.1. Separación Estricta de Capas: La clase de negocio Question en domain.model debe ser Java pura, sin ninguna anotaciones de persistencia (@Entity, @Table, @Id).

3.2. Modelo Relacional JPA: En el paquete infrastructure.jpa.record, crear la entidad QuestionRecord anotada con @Entity y definir la relación @OneToMany(cascade = CascadeType.ALL) hacia AnswerOptionRecord.

3.3. Mapeador y Repositorio: Implementar el mapeador bidireccional QuestionDomainMapper para convertir entre Question y QuestionRecord. Completar QuestionRepositoryJpaImpl inyectando SpringQuestionDataRepository para persistir la pregunta en base de datos H2.

3. Rúbrica de Evaluación y Matriz de Pruebas

Criterio	Indicador de Logro y Evidencia Requerida	Peso
Patrones GoF (Builder + State)	Construcción fluida con validación de 4 opciones y cambio de estado Draft → PendingReview sin condicionales.	**30%**
Comunicación REST + RabbitMQ	Conexión síncrona HTTP REST entre microservicios, publicación AMQP, respuesta 202 Accepted y @RabbitListener.	**40%**
Persistencia JPA Limpia	Separación de dominio puro y @Entity JPA Record, Mapeador bidireccional y repositorio funcional en H2.	**30%**
Total	**Evaluación práctica completa en Postman y consola de RabbitMQ (http://localhost:15672).**	**100%**